编译原理
10. 活跃变量分析

rainoftime.github.io
浙江大学
计算机科学与技术学院

Content

1. Introduction
2. Lexical Analysis
3. Parsing
4. Abstract Syntax
5. Semantic Analysis
6. Activation Record
7. Translating into Intermediate Code
8. Basic Blocks and Traces
9. Instruction Selection
10. **Liveness Analysis**
11. Register Allocation
13. Garbage Collection
14. Object-oriented Languages
18. Loop Optimizations

Outline

1. Dataflow Analysis
2. Liveness Analysis
3. More Discussions

1. Dataflow Analysis

- Compiler Optimizations
- Dataflow Analysis

Granularities of Compiler Optimizations

- **Local**
  - Work on a single basic block, or consider multiple blocks but less than whole procedure
- **Intraprocedural (or "global')**
  - Create on an entire procedure
- **Interprocedural (or "whole-program")**
  - Operate on > 1 procedure, up to whole program
  - Sometimes, at link time (LTO, link time optimization)

Regardless of Optimization Level

1. Analyze program to gather "facts"

– Perform "program analysis" on the program's **IR**

- A pointer v points to S
- Ranges of integer x
- value defined at d: x = …
  may be used at u: … x …

2. Apply transformation (e.g., optimizations)

- Constant folding, dead code elimination, …
- Loop-invariant code motion, register allocation,…

IRs for Analyses and Optimizations

At each of these scopes, the compiler uses various IRs for performing the analyses and optimizations

- **Local**
  - E.g., dependence graph
- **Intraprocedural (or global)**
  - E.g., control-flow graph for dataflow analysis
- **Interprocedural (or who-program)**
  - E.g., Call graph, ICFG, SDG

1. Dataflow Analysis

- Compiler Optimizations
- Dataflow Analysis

The Control Flow Graph (CFG)

- **Control Flow Graph:** A directed graph
  - Nodes represent statements
  - Edges represent control flow

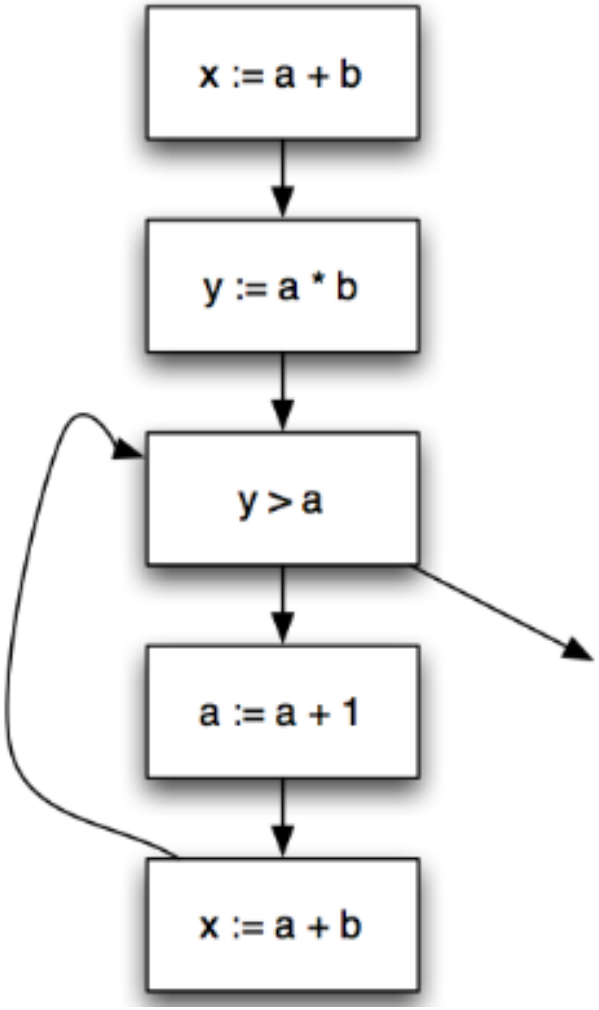
- Statements may be
  - Assignments x := y op z or x := op z
  - Copy statements x := y
  - Branches goto L or if x relop y goto L
  - etc.

Frances Allen (1932–2020), (1st woman to receive Turing Award in 2006)

Example: Control Flow Graph (CFG)

- node: a statement.
- edge: control flow.

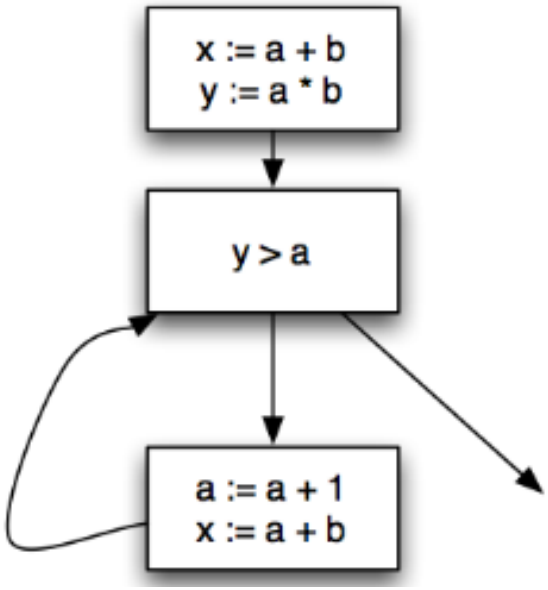
```
x := a + b;
y := a * b;
while (y > a) {
    a := a + 1;
    x := a + b;
}
```



Variants on Control Flow Graph

- We can group statements into a single basic block
  - A **basic block**: a sequence of instructions with unique entry and exit (recall Chapter 8)

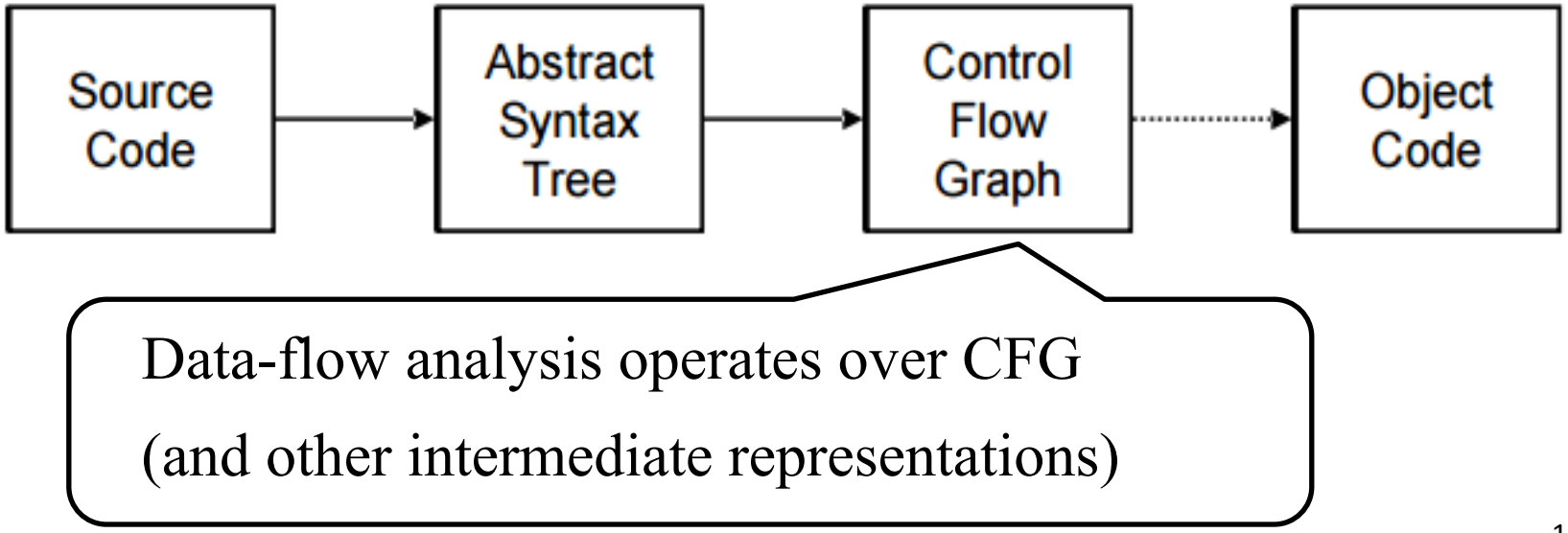
```
x := a + b;
y := a * b;
while (y > a + b) {
    a := a + 1;
    x := a + b;
}
```



We'll use single-statement basic blocks in this lecture

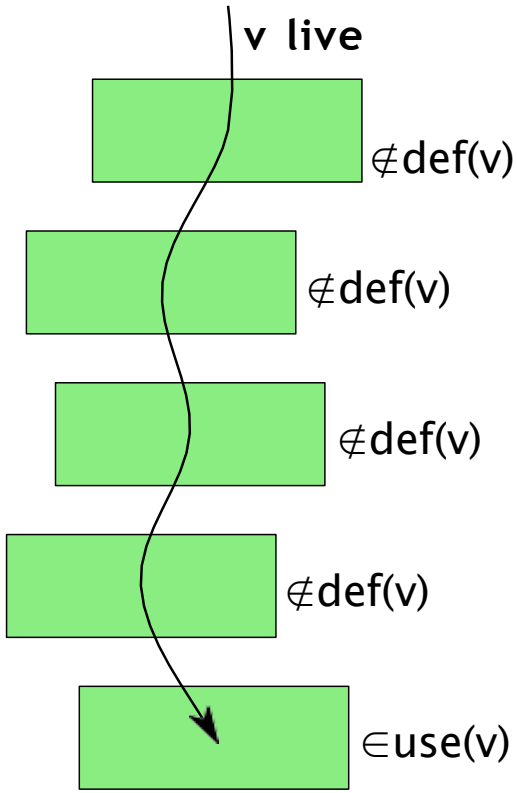
Dataflow Analysis

- A framework for deriving information about the dynamic behavior of a program without running it
  - "Dataflow facts" : liveness, types, …
  - Applications in optimization/verification/…



Example: Liveness of Variables

- **Definition**: A variable x is live at statement s if
  1. There exists a statement s' that uses x
  2. There is a path from s to s'
  3. That path has no intervening assignment to x



Undecidability of Program Analysis

- We cannot precisely compute live variables. To see why, consider the following code:

```
1: x = 10; // is x live here?
2: f();
3: return x;
```

- It seems to be obvious that x is live after Line 1
- However, suppose that f() never returns!
  - In that case the value of x is not needed
  - In other words, x is live if f() halts

Since the **halting problem** is undecidable, so is the live variable problem (if we want to obtain precise results)!

Undecidability of Program Analysis

- In fact, all interesting facts about programs (by a reasonable definition of "interesting") are undecidable for similar reasons (**Rice Theorem**)

- No compiler can really tell if a variable's value is truly needed – whether the variable is truly live.

```
x = y; // is x live here?
f();  // does f halt??
return x;
```

Approximating the "Exact Solutions"

- When we do (static) program analysis, we are usually **approximating** facts about the programs
  - E.g., assume that any if-branch goes both ways.

- For **liveness analysis**: **overapproximates** the true set of live variables by finding all of the variables that *may* be needed later

Typically, formulated as a **dataflow analysis** problem and solved by a dataflow analysis framework.

Taxonomy of Dataflow Analysis （供参考）

- **Forward vs. Backward analysis**
  - **Forward**: property of a node depends on those of its predecessors. E.g., available expressions
  - **Backward**: property of a node depends on those of its successors. E.g. live variables.

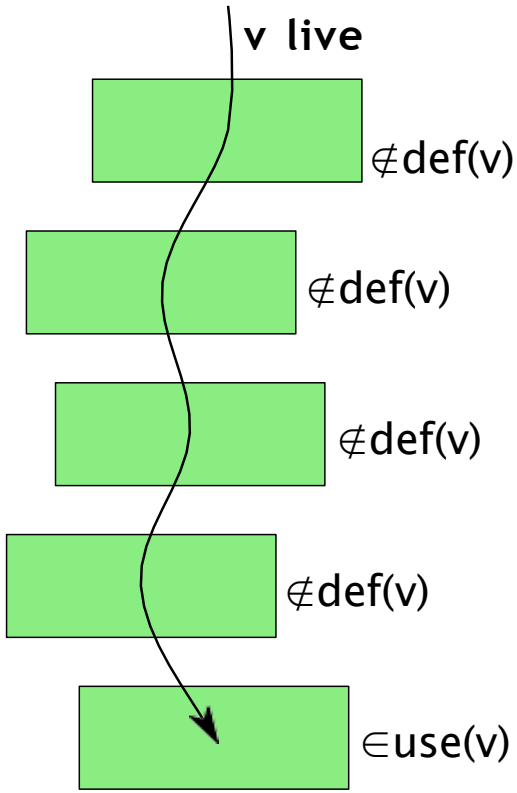
- **Must vs. May analysis**
  - **Must**: property must be true in all successors/predecessors of a node to be true. E.g. available expressions.
  - **May:** property must be true in at least one successor/predecessor of a node to be true. E.g. liveness

2. Liveness Analysis

- **Live Variables**
- **Dataflow Equations for Liveness**
- **Solving the Equations**

Recap: Live Variables

- **Definition**: A variable x is live at statement s if
  1. There exists a statement s' that uses x
  2. There is a path from s to s'
  3. That path has no intervening assignment to x



Motivating Example: Register Allocation

- Low level IRs after instruction selection assume an infinite number of "abstract registers"
  - Good for code generations
  - But bad for execution on a real machine

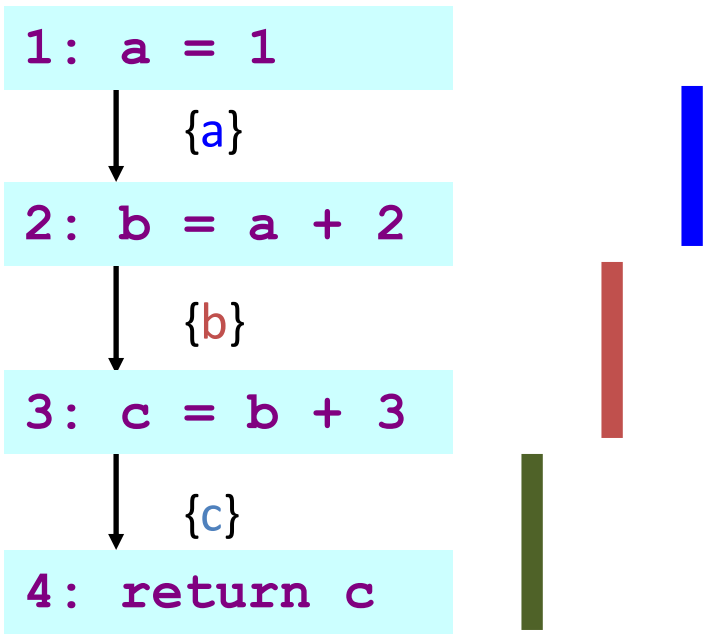
- The goal of register allocation: put infinite variables into a finite machine registers
  - Typically, need a **liveness analysis**

Motivating Example: Register Allocation

```
1: a = 1

2: b = a + 2

3: C = b + 3

4: return c
```

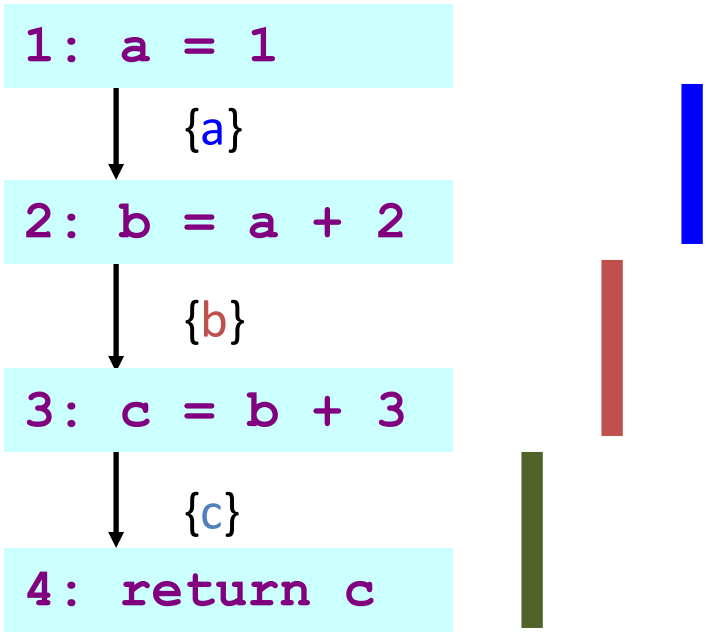
- Three variables: a, b, and c.
- And assume that the target machine has only one register: r.
- Is it possible to put all three variables “a”, “b” and “c” in register “r”?

Motivating Example: Register Allocation



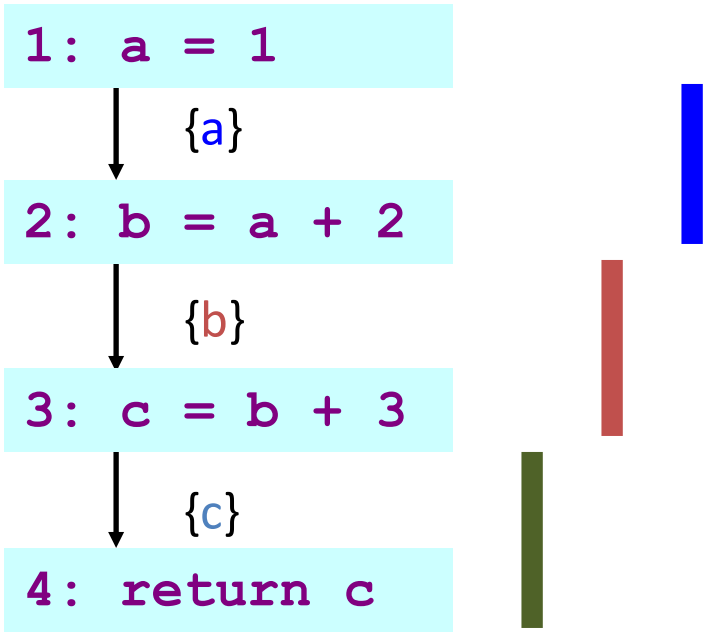
- Calculate which variable is "live" at a given program point.
- The "liveness" info. gives **live ranges**.
- Live ranges of
  - a: 1->2
  - b: 2 -> 3
  - c: 3 -> 4

Motivating Example: Register Allocation



- Calculate which variable is "live" at a given program point.
- The "liveness" info. gives **live ranges**.
- Live ranges of a, b, c don't overlap, thus all three variables can be put into ONE register!

Motivating Example: Register Allocation



- Register allocation:

a => r

b => r

c => r

- "Code rewriting":

r = 1

r = r + 2

r = r + 3

return r

More Application of Liveness Information

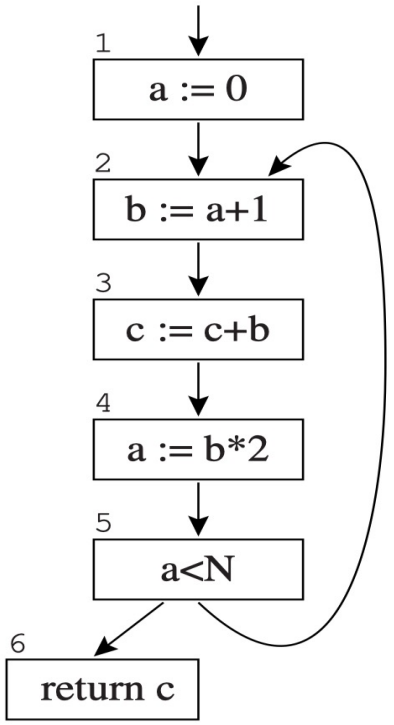
- **Code Optimizations**
  - Remove unused assignments
- **IR Construction**
  - Optimize the construction of SSA
- **Security/Reliability**
  - Detect the use of uninitialized variables

3. Liveness Analysis

- Live Variables
- Dataflow Equations for Liveness
- Solving the Equations

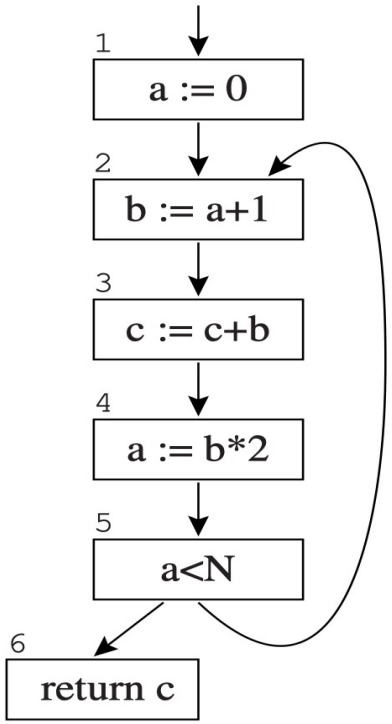
Flow Graph Terminologies

- Each **node** = one statement
  - **out-edges**: lead to successor nodes
  - **in-edges**: come from predecessor nodes
  - **pred[n]**: the predecessors of node n
  - **succ[n]**: the successors of node n
- Example
  - out-edges of node 5: **?**
  - succ[5] = **?**
  - in-edges of 2 are **?**
  - pred[2] = **?**



Flow Graph Terminologies

- Each **node** = one statement
  - **out-edges**: lead to successor nodes
  - **in-edges**: come from predecessor nodes
  - **pred[n]**: the predecessors of node n
  - **succ[n]**: the successors of node n
- Example
  - out-edges of node 5: 5->6, 5-2
  - succ[5] = {2, 6}
  - in-edges of 2 are 5->2, 1->2
  - pred[2] = {1, 5}



Liveness info. "**flows**" around the edges of the CFG

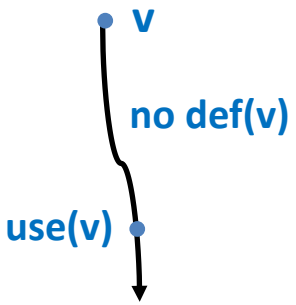
Dataflow Analysis: A General Framework

1. **Set up local equations at each flow graph node**
   - For a node n, we write
     - **in[n]**: facts that are true on all in-edges to the node
     - **out[n]**: facts true on all out-edges
   - Define **transfer functions** that transfer information from one node to another

2. **Solve the equations for the desired information**
   - Update **in[n]** and **out[n]** until convergence

Recap: Live Variables

- **Live variable**: a variable x is live at statement s if
  1. There exists a statement s' that **uses** x
  2. There is a path from s to s'
  3. That path has no intervening **assignment/definition** to x



What are "uses" and "defs" and how to define in and out

Defs and Uses

- Def: Assignment to a variable or temporary
- Use: An occurrence of a variable on the right-hand side of an assignment (or other expressions)

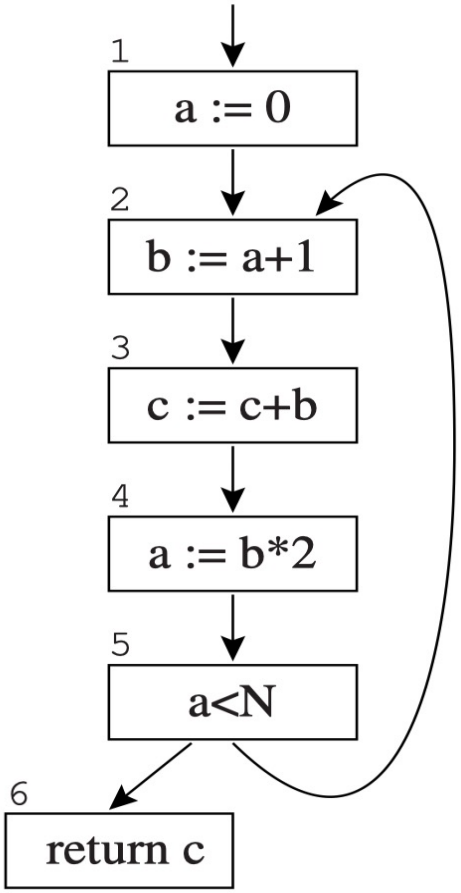
```
x = y + z  // y and z are used, x is defined
 if (a < b) // a and b are used, nothing defined
return c    // c is used, nothing defined
```

For each node n in the flow graph, we track:

- **def** (n): Set of variables defined at n
- **use(n)**: Set of variables used at n

Example: Defs and Uses

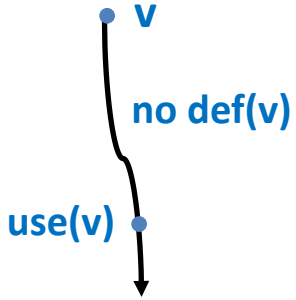
```
1: a ← 0
2: b ← a + 1
3: c ← c + b
4: a ← b * 2
5: if a < N
goto L1
6: return c
```



Node	use(n)	def(n)
1	{ }	{a}
2	{a}	{b}
3	{b, c}	{c}
4	{b}	{a}
5	{a, N}	{ }
6	{c}	{ }

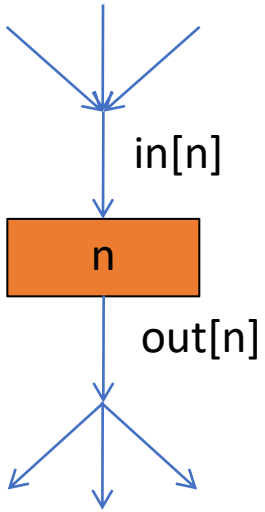
Liveness Facts

- A variable is **live on an edge**: there is a directed path from that edge to a **use** of the variable that does not go through any **def**.



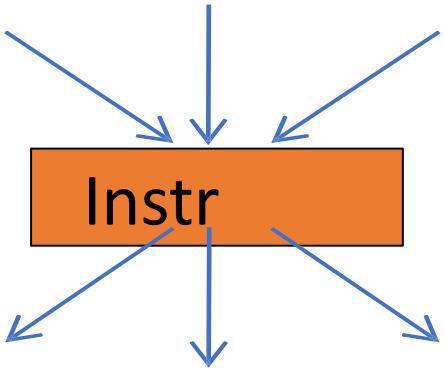
- **Live-in**: a variable is **live-in** at a node if it is live on any of the in-edges of that node;

- **Live-out**: A variable is **live-out** at a node if it is live on any of the out-edges of the node.



Liveness Facts

- in[n] : the **live-in** set of node n (variables live at entry to node n)
- out[n] : the **live-out** set of node n (variables live at exist from node n)



Live: a, b

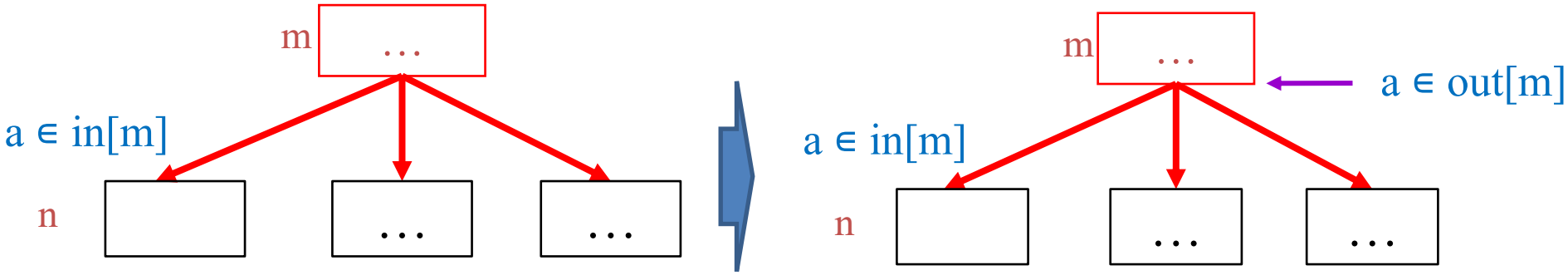
Live: b, d, e

How to calculate in[n] and out[n]

Rules for Liveness

- **Rule 1: Live-in propagates to predecessors**
  - **If a ∈ in[n], then for ∀m ∈ pred[n], a ∈ out[m]**

  If a variable is *live-in* at a node n, it is *live-out* at all nodes m in ***pred*[n]** (predecessors of n)

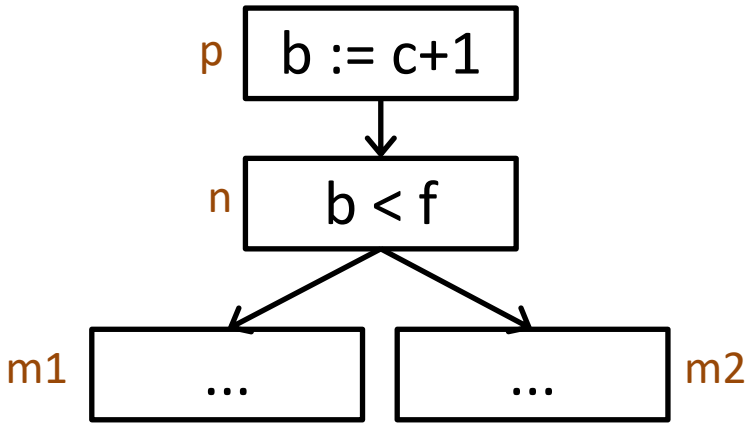


Rules for Liveness

- **Rule 1: Live-in propagates to predecessors**
  - **If a ∈ in[n], then for ∀m ∈ pred[n], a ∈ out[m]**

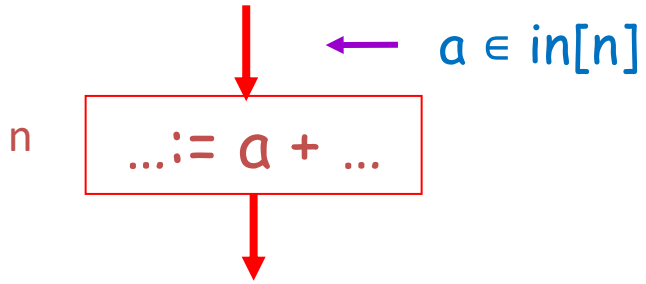
    If a variable is *live-in* at a node n, it is *live-out* at all nodes m in ***pred*[n]** (predecessors of n)

- Example: given in[m1] = {d}, in[m2] = {e}
  - **out[n] = {d, e}**

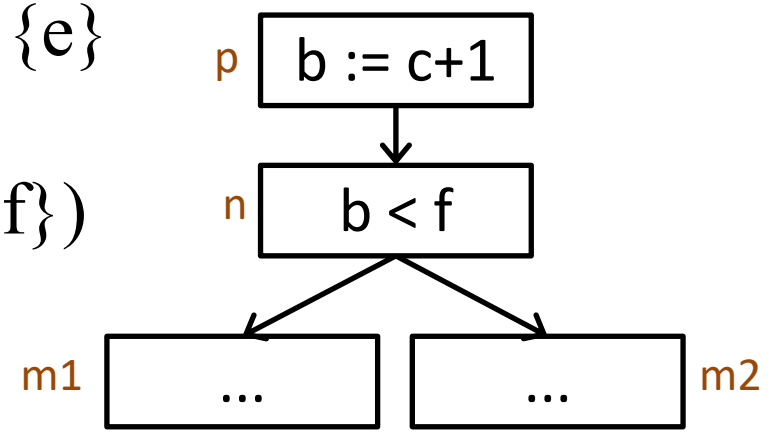


Rules for Liveness

- **Rule 2:** Use implies live-in
  - If a ∈ use[n], then a ∈ in[n]

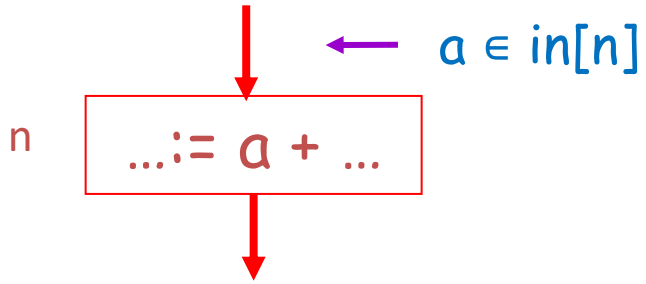


- Given: in[m1] = {d}, in[m2] = {e}
  - out[n] = {d, e}
  - **in[n] = {b, f}** (Use(n) = {b, f})

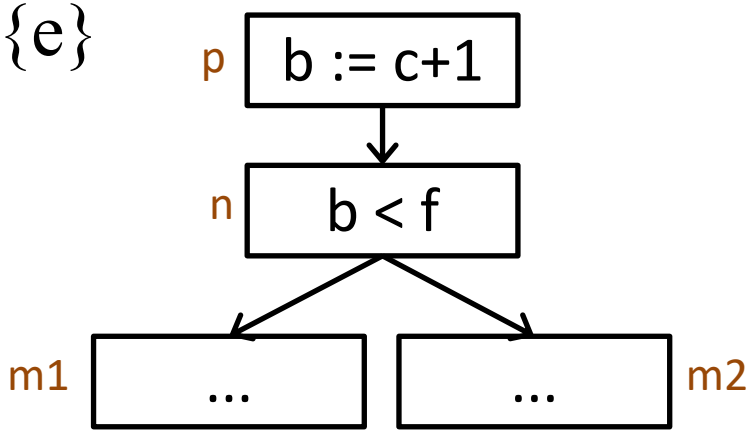


Rules for Liveness

- **Rule 2: Use implies live-in**
  - **If a ∈ use[n], then a ∈ in[n]**



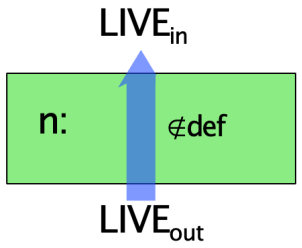
- Given: in[m1] = {d}, in[m2] = {e}
  - out[n] = {d, e}
  - **in[n] = {b, f, …?}**



Other variables belong to in[n]?
Can only look at the uses in node n?

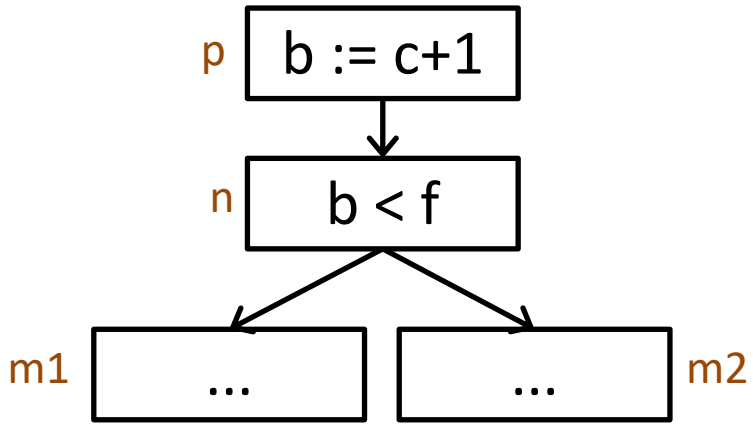
Rules for Liveness

- **Rule 3: Live-out + not-defined → live-in**
  - **If a ∈ out[n] and a ∉ def[n], then a ∈ in[n]**



Suppose: in[m1] = {d}, in[m2] = {e}

- out[n] = {d, e}
- **in[n] = {b, f, d, e}**
- out[p] = ?
- in[p] = ?

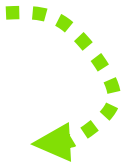


Rules for Liveness

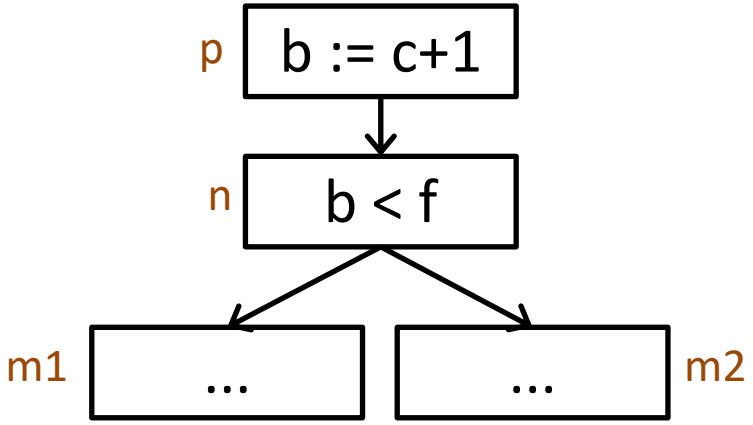
- **Rule 1: If a ∈ in[n], then for ∀m ∈ pred[n], a ∈ out[m]**
- **Rule 2: If a ∈ use[n], then a ∈ in[n]**
- **Rule 3: If a ∈ out[n] and a ∉ def[n], then a ∈ in[n]**

Suppose: in[m1] = {d}, in[m2] = {e}

- out[n] = {d, e}
- in[n] = {b, f, d, e}
- **out[p] = {b, f, d, e}**
- in[p] = ?



Rule 1



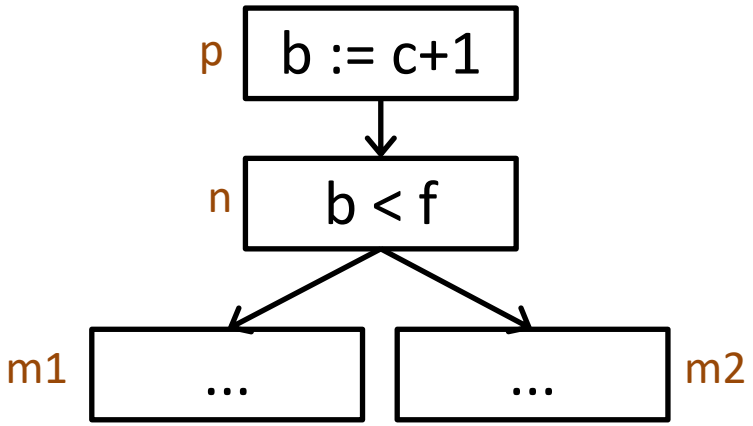
Rules for Liveness

- **Rule 1: If a ∈ in[n], then for ∀m ∈ pred[n], a ∈ out[m]**
- **Rule 2: If a ∈ use[n], then a ∈ in[n]**
- **Rule 3: If a ∈ out[n] and a ∉ def[n], then a ∈ in[n]**

Suppose: in[m1] = {d}, in[m2] = {e}

- out[n] = {d, e}
- in[n] = {b, f, d, e}
- out[p] = {b, **f, d, e**}
- **in[p] = {f, d, e}**

Rule 3

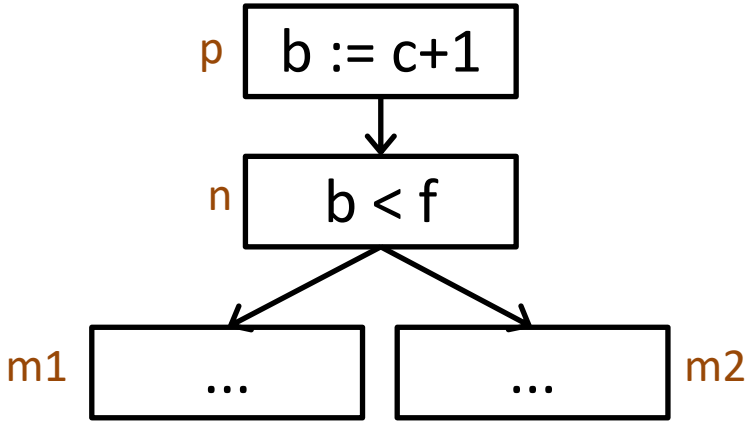


Rules for Liveness

- **Rule 1: If a ∈ in[n], then for ∀m ∈ pred[n], a ∈ out[m]**
- **Rule 2: If a ∈ use[n], then a ∈ in[n]**
- **Rule 3: If a ∈ out[n] and a ∉ def[n], then a ∈ in[n]**

Suppose: in[m1] = {d}, in[m2] = {e}

- out[n] = {d, e}
- in[n] = {b, f, d, e}
- out[p] = {b, f, d, e}
- **in[p] = {c, f, d, e}**



Summary: Rules of Liveness

- **Rule 1:** Live-in propagates to predecessors
  - If $a \in in[n]$, then for $\forall m \in pred[n]$, $a \in out[m]$
- **Rule 2:** Use implies live-in
  - If $a \in use[n]$, then $a \in in[n]$
- **Rule 3:** Live-out + not-defined → live-in
  - If $a \in out[n]$ and $a \notin def[n]$, then $a \in in[n]$

Takeaway message: "If someone needs *v*'s value after *n*, and *n* doesn't provide it, then *v*'s value is needed before *n* too."

From Rules to Dataflow Equations

- **Rule 1:** If $a \in in[n]$, then for $\forall m \in pred[n]$, $a \in out[m]$
- **Rule 2:** If $a \in use[n]$, then $a \in in[n]$
- **Rule 3:** If $a \in out[n]$ and $a \notin def[n]$, then $a \in in[n]$

$$in[n] = use[n] \cup (out[n] - def[n])$$

$$out[n] = \bigcup_{s \in succ[n]} in[s]$$

"What's needed **before** node *n* executes?"

- Variables this node reads (*use*)
- Variables needed after (*out*) that this node doesn't write (*def*)

From Rules to Dataflow Equations

- **Rule 1:** If $a \in in[n]$, then for $\forall m \in pred[n]$, $a \in out[m]$
- **Rule 2:** If $a \in use[n]$, then $a \in in[n]$
- **Rule 3:** If $a \in out[n]$ and $a \notin def[n]$, then $a \in in[n]$

$$in[n] = use[n] \cup (out[n] - def[n])$$

$$out[n] = \bigcup_{s \in succ[n]} in[s]$$

"What's needed **after** node *n* executes?"

- Whatever any successor needs on entry
- Union because control could go to **any** successor

2. Liveness Analysis

- Liveness Variables
- Dataflow Equations for Liveness
- Solving the Equations

Solving Dataflow Equations

Declarative Specification

$$in[n] = use[n] \cup (out[n] - def[n])$$

$$out[n] = \bigcup_{s \in succ[n]} in[s]$$

Runnable Algorithm

for each n
  in[n] ←{}; out[n] ←{}
repeat
  for each n
    in′[n] ← in[n]; out′[n] ← out[n]
    in[n] ← use[n] ∪ (out[n] − def[n])
    out[n] ← $\cup_{s \in succ[n]}$ in[s]
until in′[n] = in[n] and out′[n] = out[n] for all n

use_B 和def_B 的值可以直接从控制流图计算出来，因此在方程中作为已知量

Solving Dataflow Equations

```
for each n
   in[n] ←{}; out[n] ←{}
repeat
   for each n
      in′[n] ← in[n]; out′[n] ← out[n]
      in[n] ← use[n] ∪ (out[n] − def[n])
      out[n] ← ⋃s ∈succ[n] in[s]
until in′[n] = in[n] and out′[n] = out[n] for all n
```

- **Initialize all sets to empty**
  - Start with a "rough" approximation to the answer
- Repeatedly apply dataflow equations
- Continue until no changes (fixed point)

Solving Dataflow Equations

```
for each n
  in[n] ←{}; out[n] ←{}
repeat
  for each n
    in′[n] ← in[n]; out′[n] ← out[n]
    in[n] ← use[n] ∪ (out[n] − def[n])
    out[n] ← ⋃s ∈succ[n] in[s]
until in′[n] = in[n] and out′[n] = out[n] for all n
```

- Initialize all sets to empty
- **Repeatedly apply dataflow equations**
  - Each iteration adds new variables to in[n] and out[n] monotonically
  - The sizes of in[n] and out[n] are bounded
- Continue until no changes (fixed point)

Solving Dataflow Equations

```
for each n
  in[n] ←{}; out[n] ←{}
repeat
  for each n
    in′[n] ← in[n]; out′[n] ← out[n]
    in[n] ← use[n] ∪ (out[n] − def[n])
    out[n] ← ⋃s ∈succ[n] in[s]
until in′[n] = in[n] and out′[n] = out[n] for all n
```

- Initialize all sets to empty
- Repeatedly apply dataflow equations
  - Each iteration adds new variables to in[n] and out[n] monotonically
  - The sizes of in[n] and out[n] are bounded
- **Continue until no changes (fixed point)**

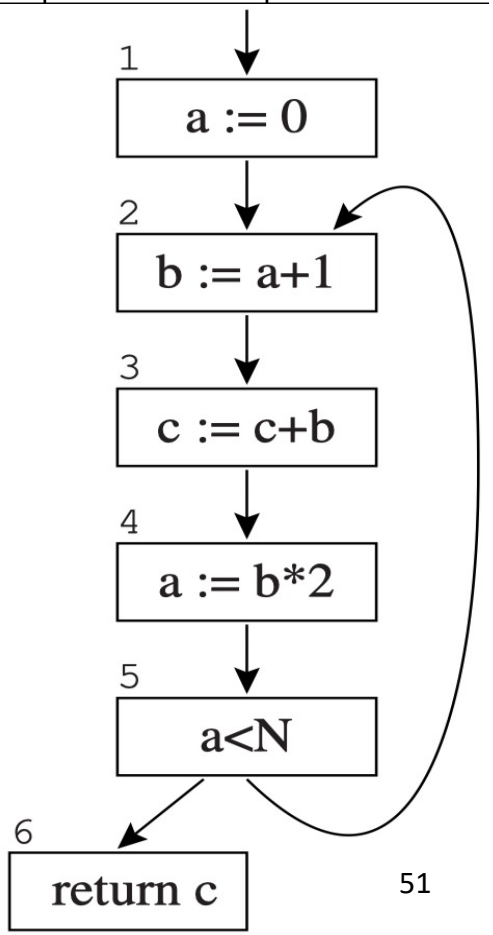
Example: Calculation of Liveness

- **Strategy**: following forward control-flow edges

	use	def	1st in	1st out	2nd in	2nd out	3rd in	3rd out	4th in	4th out	5th in	5th out	6th in	6th out	7th in	7th out
1		a														
2	a	b														
3	bc	c														
4	b	a														
5	a															
6	c															

$$in[n] = use[n] \cup (out[n] - def[n])$$

$$out[n] = \bigcup_{s \in succ[n]} in[s]$$



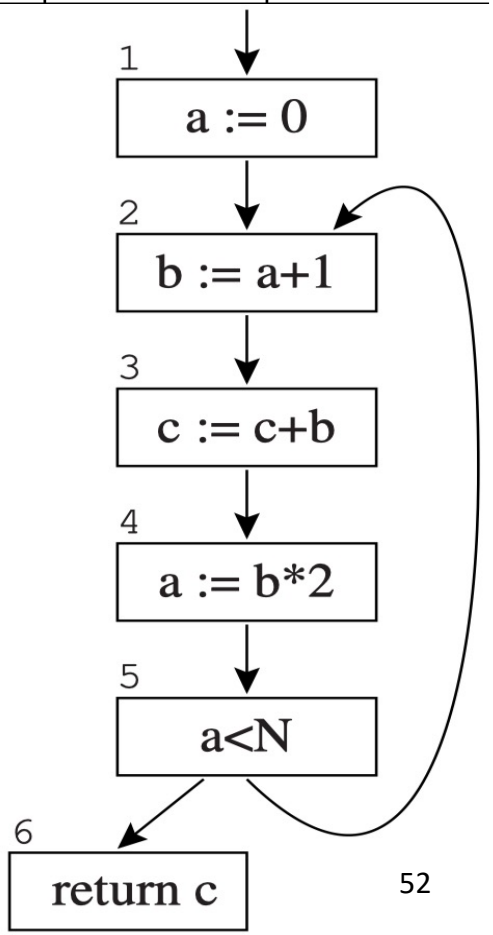
Example: Calculation of Liveness

- **Strategy**: following forward control-flow edges

	use	def	1st in	1st out	2nd in	2nd out	3rd in	3rd out	4th in	4th out	5th in	5th out	6th in	6th out	7th in	7th out
1		a														
2	a	b	a													
3	bc	c														
4	b	a														
5	a															
6	c															

$$in[n] = use[n] \cup (out[n] - def[n])$$

$$out[n] = \bigcup_{s \in succ[n]} in[s]$$



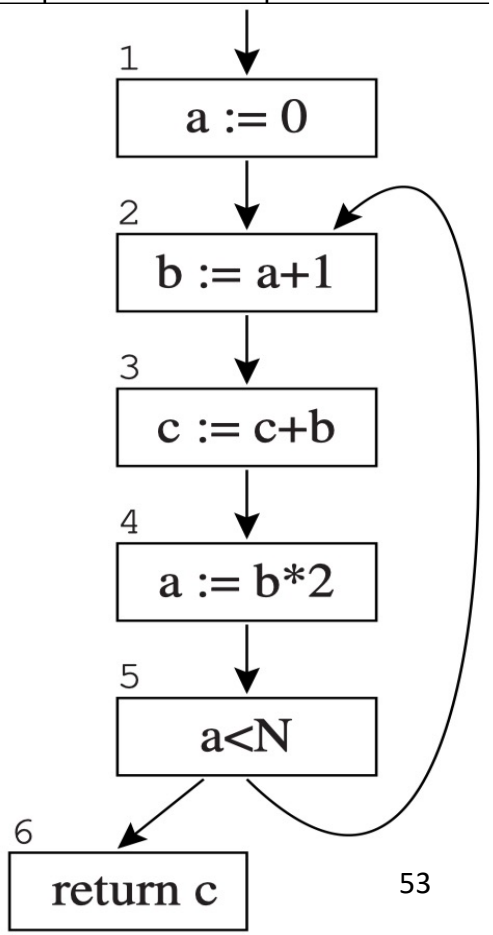
Example: Calculation of Liveness

- **Strategy**: following forward control-flow edges

	use	def	1st in	1st out	2nd in	2nd out	3rd in	3rd out	4th in	4th out	5th in	5th out	6th in	6th out	7th in	7th out
1		a														
2	a	b	a													
3	bc	c	bc													
4	b	a														
5	a															
6	c															

$$in[n] = use[n] \cup (out[n] - def[n])$$

$$out[n] = \bigcup_{s \in succ[n]} in[s]$$



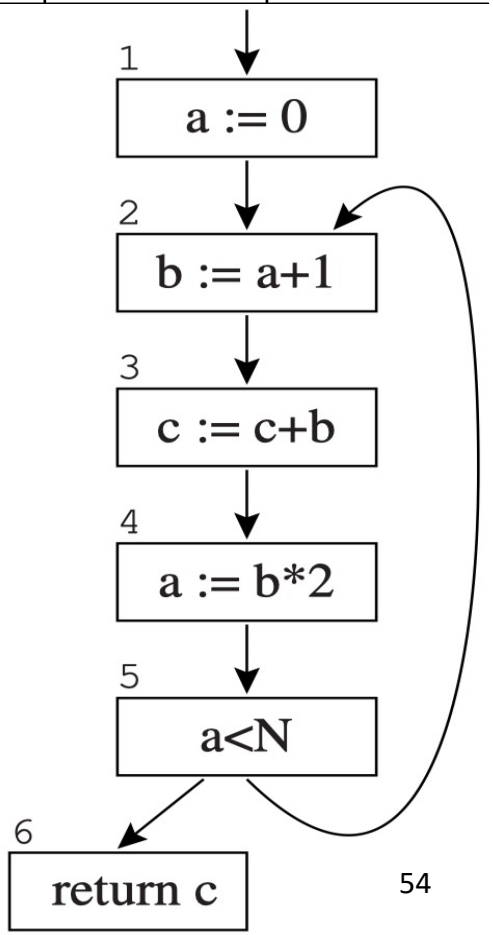
Example: Calculation of Liveness

- **Strategy**: following forward control-flow edges

	use	def	1st in	1st out	2nd in	2nd out	3rd in	3rd out	4th in	4th out	5th in	5th out	6th in	6th out	7th in	7th out
1		a														
2	a	b	a													
3	bc	c	bc													
4	b	a	b													
5	a															
6	c															

$$in[n] = use[n] \cup (out[n] - def[n])$$

$$out[n] = \bigcup_{s \in succ[n]} in[s]$$



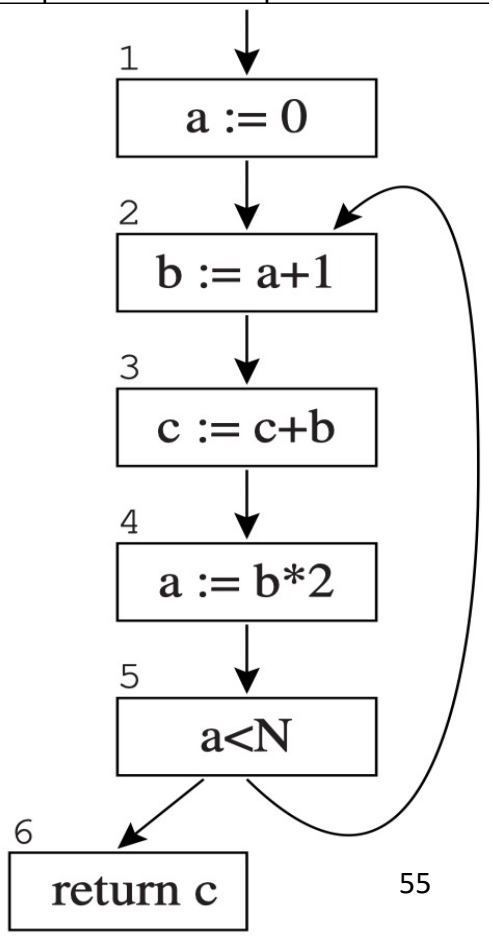
Example: Calculation of Liveness

- **Strategy**: following forward control-flow edges

	use	def	1st in	1st out	2nd in	2nd out	3rd in	3rd out	4th in	4th out	5th in	5th out	6th in	6th out	7th in	7th out
1		a														
2	a	b	a													
3	bc	c	bc													
4	b	a	b													
5	a		a													
6	c															

$$in[n] = use[n] \cup (out[n] - def[n])$$

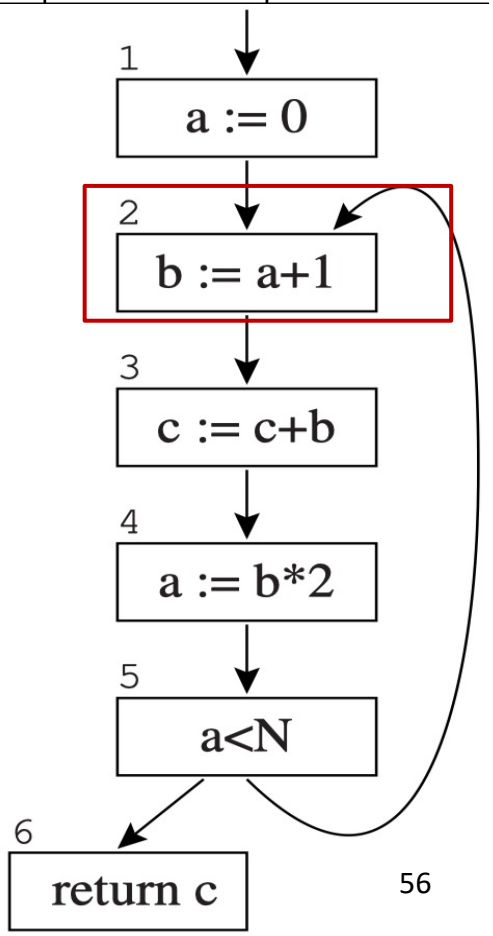
$$out[n] = \bigcup_{s \in succ[n]} in[s]$$



Example: Calculation of Liveness

- **Strategy**: following forward control-flow edges

	use	def	1st in	1st out	2nd in	2nd out	3rd in	3rd out	4th in	4th out	5th in	5th out	6th in	6th out	7th in	7th out
1		a														
2	a	b	a													
3	bc	c	bc													
4	b	a	b													
5	a		a	a												
6	c															



$$in[n] = use[n] \cup (out[n] - def[n])$$

$$out[n] = \bigcup_{s \in succ[n]} in[s]$$

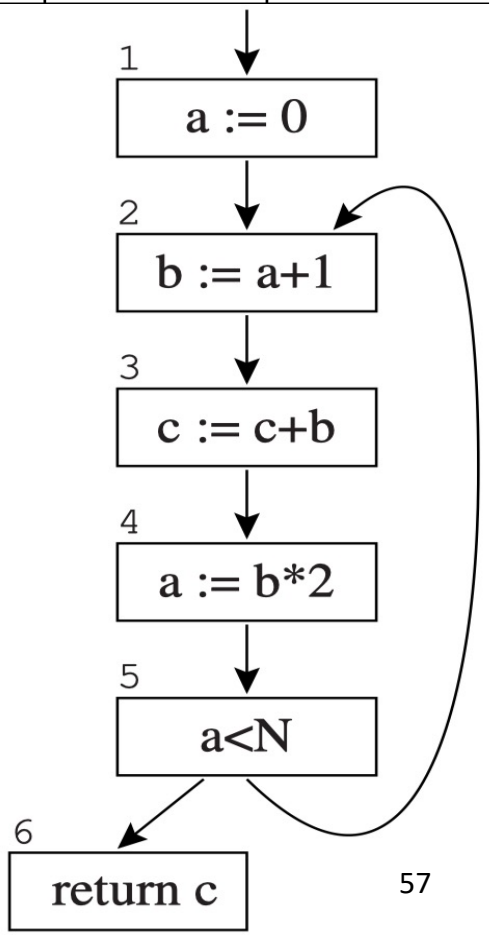
Example: Calculation of Liveness

- **Strategy**: following forward control-flow edges

	use	def	1st in	1st out	2nd in	2nd out	3rd in	3rd out	4th in	4th out	5th in	5th out	6th in	6th out	7th in	7th out
1		a														
2	a	b	a													
3	bc	c	bc													
4	b	a	b													
5	a		a	a												
6	c		c													

$$in[n] = use[n] \cup (out[n] - def[n])$$

$$out[n] = \bigcup_{s \in succ[n]} in[s]$$



Example: Calculation of Liveness

- **Strategy**: following forward control-flow edges

			1st		2nd		3rd		4th		5th		6th		7th	
	use	def	in	out	in	out	in	out	in	out	in	out	in	out	in	out
1		a				a		a		ac	c	ac	c	ac	c	ac
2	a	b	a		a	bc	ac	bc	ac	bc	ac	bc	ac	bc	ac	bc
3	bc	c	bc		bc	b	bc	b	bc	b	bc	b	bc	bc	bc	bc
4	b	a	b		b	a	b	a	b	ac	bc	ac	bc	ac	bc	ac
5	a		a	a	a	ac	ac	ac	ac	ac	ac	ac	ac	ac	ac	ac
6	c		c		c		c		c		c		c		c	

Takes **7 iterations** to converge with forward order. Slow!

How to Speed Up the Iteration?

- **Observation**: the only way information propagates from one node to another is using: $out[n] := \cup_{s \in succ[n]} in[s]$
  - The only rule that involves more than one node
  - Liveness analysis is a "backward analysis"

$$in[n] = use[n] \cup (out[n] - def[n])$$

$$out[n] = \bigcup_{s \in succ[n]} in[s]$$

- Idea for an improved version of the algorithm:
  - Keep track of which node's successors have changed
  - Compute in the opposite order of control flows
  - For each node, compute out sets before in sets

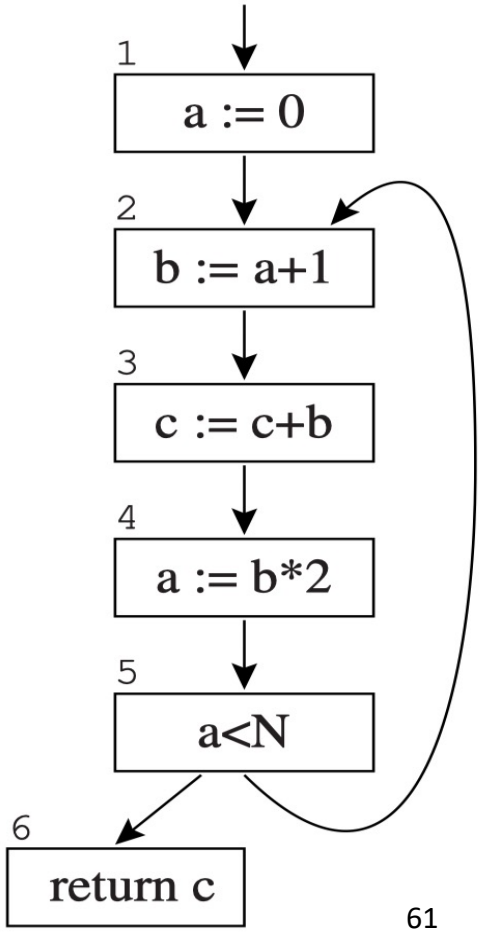
Example: Calculation of Liveness, Revisited

- out[n] is computed from in[s], in[n] is computed from out[n]
- **Strategy**: computing in the opposite order (from 6 to 1, from out to in)

	use	def	1st out	1st in	2nd out	2nd in	3rd out	3rd in
6	c							
5	a							
4	b	a						
3	bc	c						
2	a	b						
1		a						

$$in[n] = use[n] \cup (out[n] - def[n])$$

$$out[n] = \bigcup_{s \in succ[n]} in[s]$$



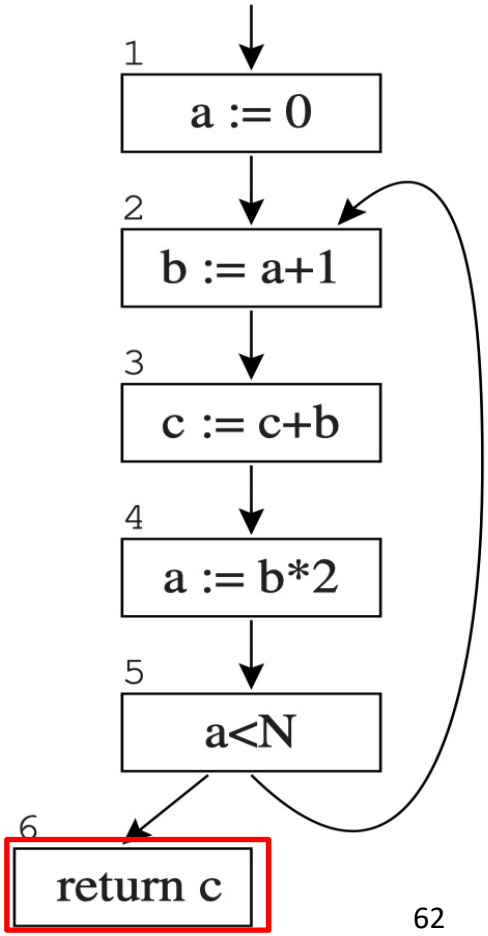
Example: Calculation of Liveness, Revisited

- out[n] is computed from in[s], in[n] is computed from out[n]
- **Strategy**: the opposite order (from 6 to 1, from out to in)

	use	def	1st out	1st in	2nd out	2nd in	3rd out	3rd in
6	c							
5	a							
4	b	a						
3	bc	c						
2	a	b						
1		a						

$$in[n] = use[n] \cup (out[n] - def[n])$$

$$out[n] = \bigcup_{s \in succ[n]} in[s]$$



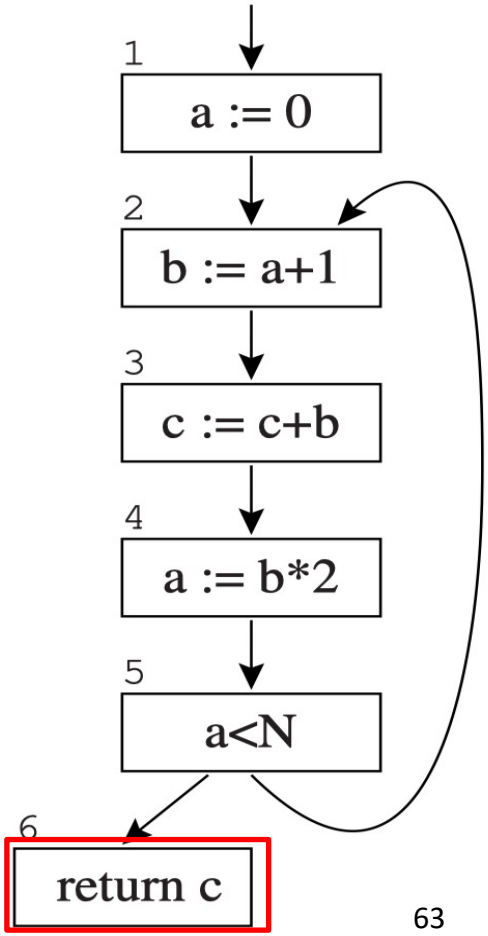
Example: Calculation of Liveness, Revisited

- out[n] is computed from in[s], in[n] is computed from out[n]
- **Strategy**: the opposite order (from 6 to 1, from out to in)

	use	def	1st out	1st in	2nd out	2nd in	3rd out	3rd in
6	c			c				
5	a							
4	b	a						
3	bc	c						
2	a	b						
1		a						

$$in[n] = use[n] \cup (out[n] - def[n])$$

$$out[n] = \bigcup_{s \in succ[n]} in[s]$$



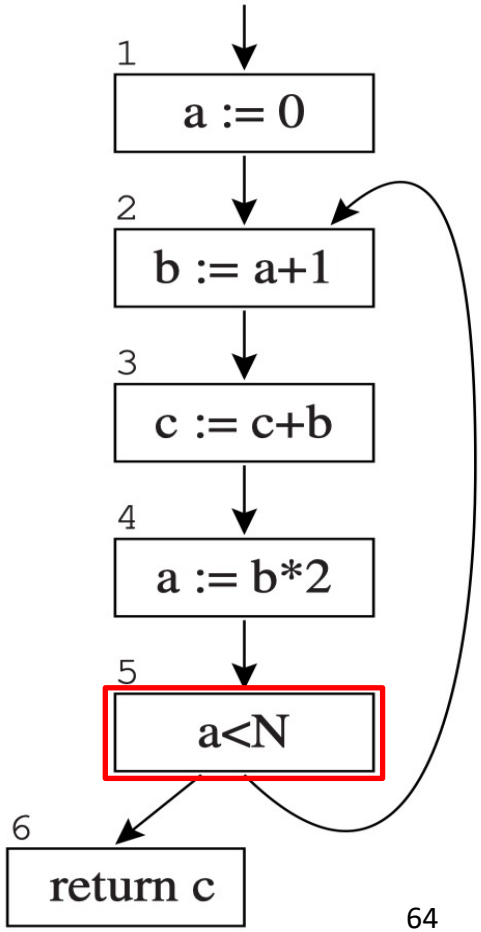
Example: Calculation of Liveness, Revisited

- out[n] is computed from in[s], in[n] is computed from out[n]
- **Strategy**: the opposite order (from 6 to 1, from out to in)

	use	def	1st out	1st in	2nd out	2nd in	3rd out	3rd in
6	c			c				
5	a		c					
4	b	a						
3	bc	c						
2	a	b						
1		a						

$$in[n] = use[n] \cup (out[n] - def[n])$$

$$out[n] = \bigcup_{s \in succ[n]} in[s]$$



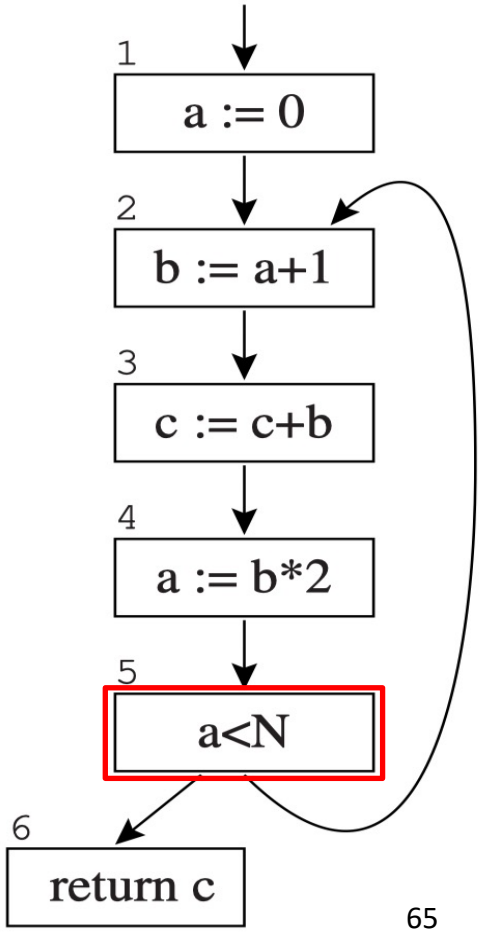
Example: Calculation of Liveness, Revisited

- out[n] is computed from in[s], in[n] is computed from out[n]
- **Strategy**: the opposite order (from 6 to 1, from out to in)

	use	def	1st out	1st in	2nd out	2nd in	3rd out	3rd in
6	c			c				
5	a		c	ac				
4	b	a						
3	bc	c						
2	a	b						
1		a						

$$in[n] = use[n] \cup (out[n] - def[n])$$

$$out[n] = \bigcup_{s \in succ[n]} in[s]$$



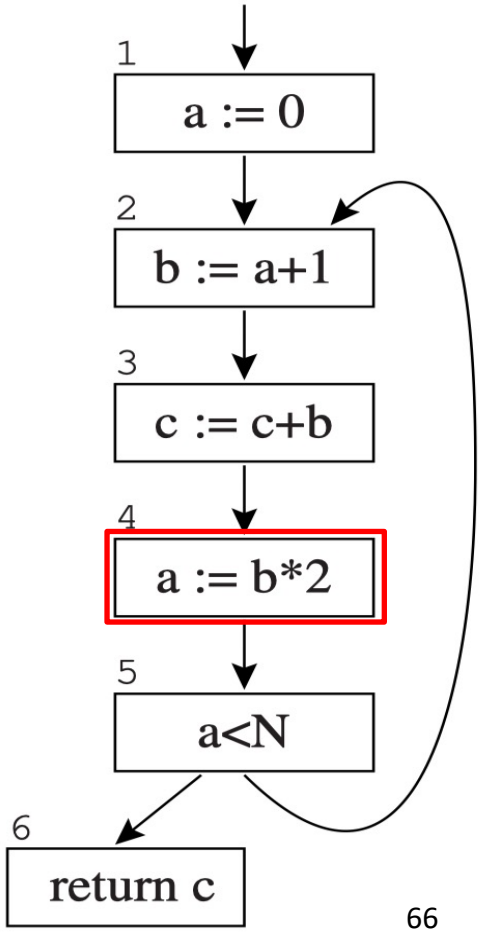
Example: Calculation of Liveness, Revisited

- out[n] is computed from in[s], in[n] is computed from out[n]
- **Strategy**: the opposite order (from 6 to 1, from out to in)

	use	def	1st out	1st in	2nd out	2nd in	3rd out	3rd in
6	c			c				
5	a		c	ac				
4	b	a	ac					
3	bc	c						
2	a	b						
1		a						

$$in[n] = use[n] \cup (out[n] - def[n])$$

$$out[n] = \bigcup_{s \in succ[n]} in[s]$$



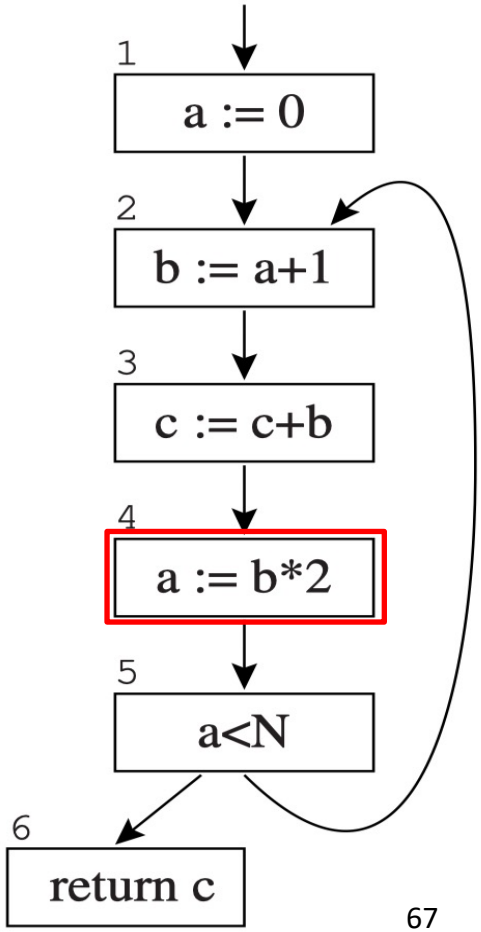
Example: Calculation of Liveness, Revisited

- out[n] is computed from in[s], in[n] is computed from out[n]
- **Strategy**: the opposite order (from 6 to 1, from out to in)

	use	def	1st out	1st in	2nd out	2nd in	3rd out	3rd in
6	c			c				
5	a		c	ac				
4	b	a	ac	bc				
3	bc	c						
2	a	b						
1		a						

$$in[n] = use[n] \cup (out[n] - def[n])$$

$$out[n] = \bigcup_{s \in succ[n]} in[s]$$



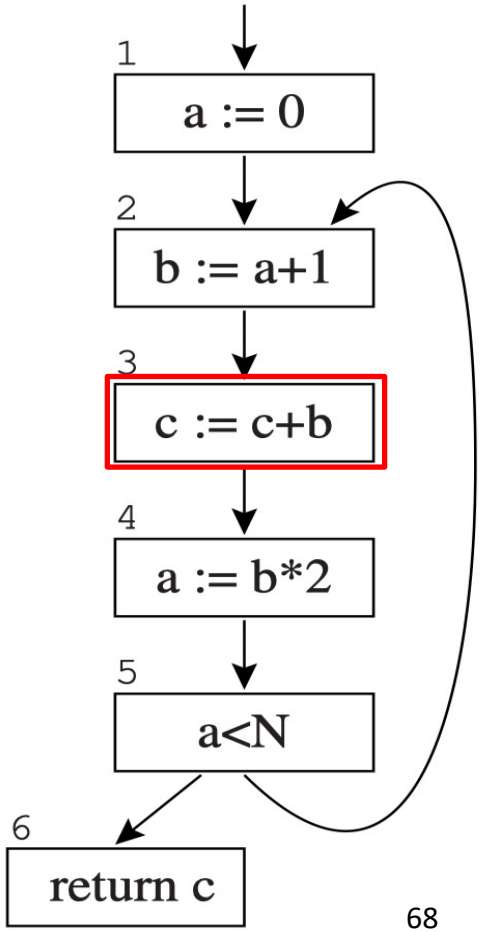
Example: Calculation of Liveness, Revisited

- out[n] is computed from in[s], in[n] is computed from out[n]
- **Strategy**: the opposite order (from 6 to 1, from out to in)

	use	def	1st out	1st in	2nd out	2nd in	3rd out	3rd in
6	c			c				
5	a		c	ac				
4	b	a	ac	bc				
3	bc	c	bc					
2	a	b						
1		a						

$$in[n] = use[n] \cup (out[n] - def[n])$$

$$out[n] = \bigcup_{s \in succ[n]} in[s]$$



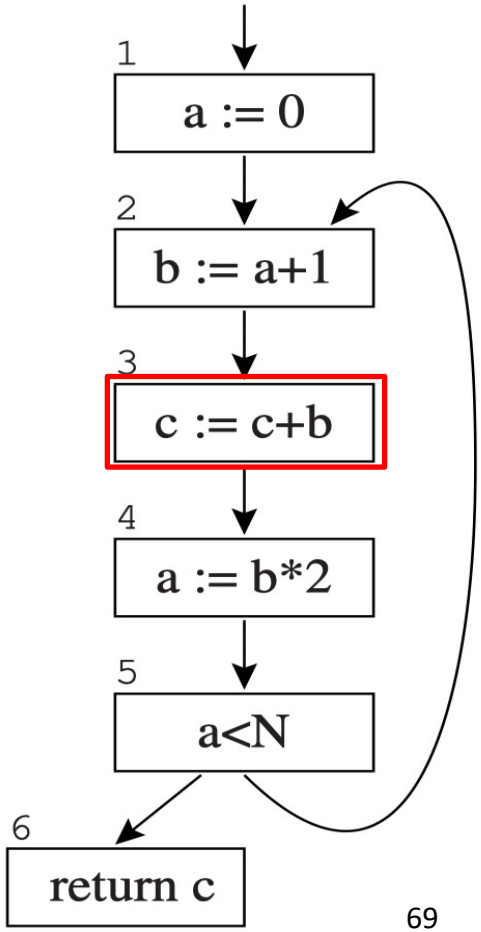
Example: Calculation of Liveness, Revisited

- out[n] is computed from in[s], in[n] is computed from out[n]
- **Strategy**: the opposite order (from 6 to 1, from out to in)

	use	def	1st out	1st in	2nd out	2nd in	3rd out	3rd in
6	c			c				
5	a		c	ac				
4	b	a	ac	bc				
3	bc	c	bc	bc				
2	a	b						
1		a						

$$in[n] = use[n] \cup (out[n] - def[n])$$

$$out[n] = \bigcup_{s \in succ[n]} in[s]$$



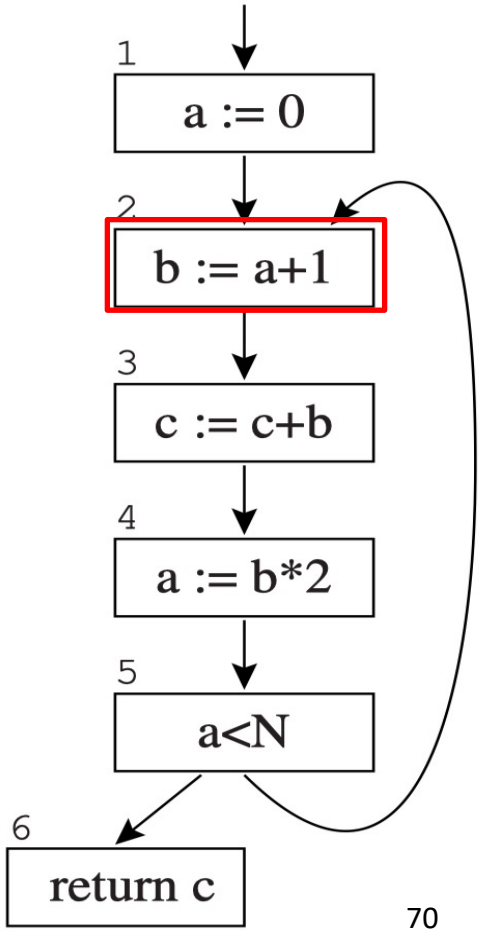
Example: Calculation of Liveness, Revisited

- out[n] is computed from in[s], in[n] is computed from out[n]
- **Strategy**: the opposite order (from 6 to 1, from out to in)

	use	def	1st out	1st in	2nd out	2nd in	3rd out	3rd in
6	c			c				
5	a		c	ac				
4	b	a	ac	bc				
3	bc	c	bc	bc				
2	a	b	bc					
1		a						

$$in[n] = use[n] \cup (out[n] - def[n])$$

$$out[n] = \bigcup_{s \in succ[n]} in[s]$$



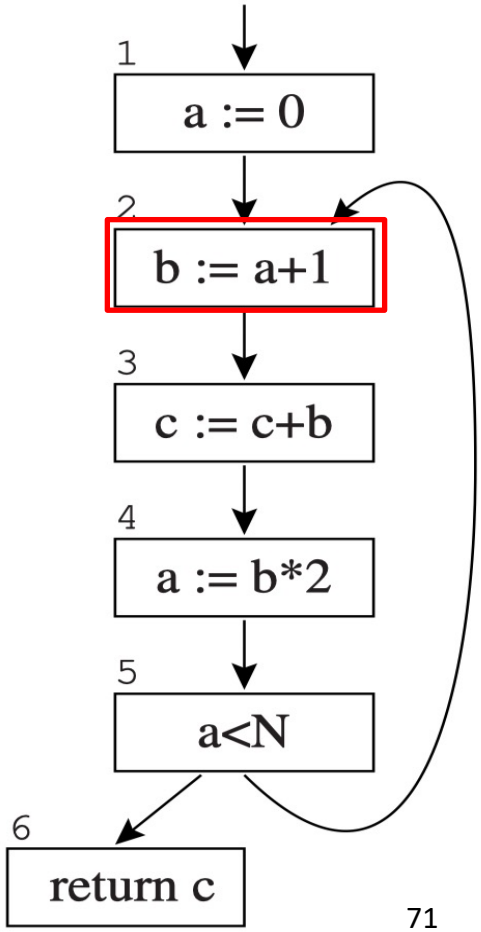
Example: Calculation of Liveness, Revisited

- out[n] is computed from in[s], in[n] is computed from out[n]
- **Strategy**: the opposite order (from 6 to 1, from out to in)

	use	def	1st out	1st in	2nd out	2nd in	3rd out	3rd in
6	c			c				
5	a		c	ac				
4	b	a	ac	bc				
3	bc	c	bc	bc				
2	a	b	bc	ac				
1		a						

$$in[n] = use[n] \cup (out[n] - def[n])$$

$$out[n] = \bigcup_{s \in succ[n]} in[s]$$



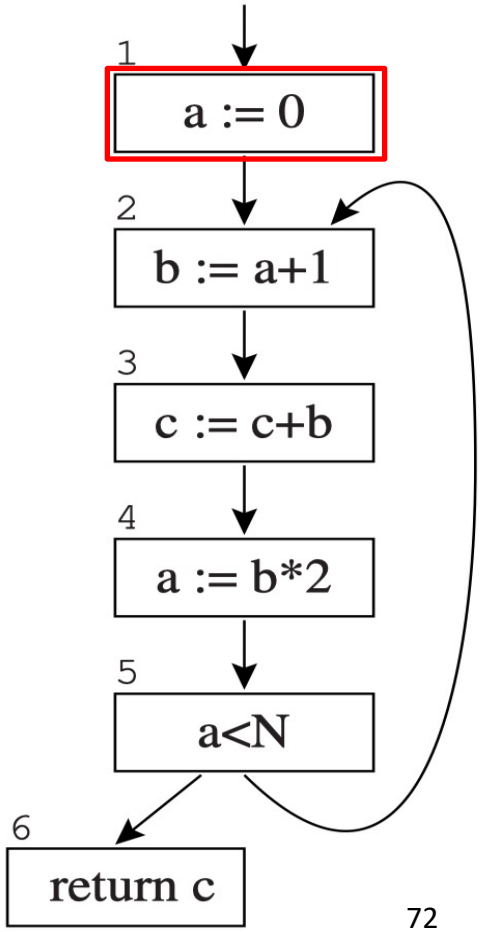
Example: Calculation of Liveness, Revisited

- out[n] is computed from in[s], in[n] is computed from out[n]
- **Strategy**: the opposite order (from 6 to 1, from out to in)

	use	def	1st out	1st in	2nd out	2nd in	3rd out	3rd in
6	c			c				
5	a		c	ac				
4	b	a	ac	bc				
3	bc	c	bc	bc				
2	a	b	bc	ac				
1		a	ac					

$$in[n] = use[n] \cup (out[n] - def[n])$$

$$out[n] = \bigcup_{s \in succ[n]} in[s]$$



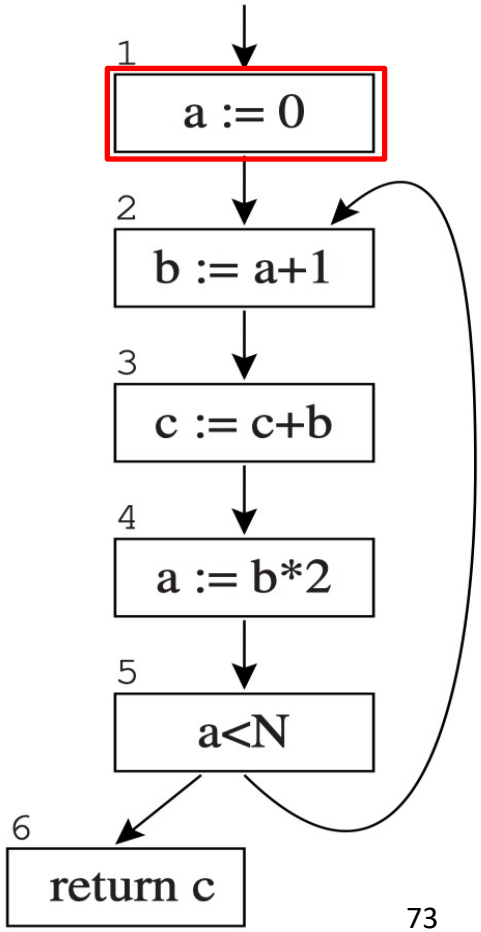
Example: Calculation of Liveness, Revisited

- out[n] is computed from in[s], in[n] is computed from out[n]
- **Strategy**: the opposite order (from 6 to 1, from out to in)

	use	def	1st out	1st in	2nd out	2nd in	3rd out	3rd in
6	c			c				
5	a		c	ac				
4	b	a	ac	bc				
3	bc	c	bc	bc				
2	a	b	bc	ac				
1		a	ac	c				

$$in[n] = use[n] \cup (out[n] - def[n])$$

$$out[n] = \bigcup_{s \in succ[n]} in[s]$$



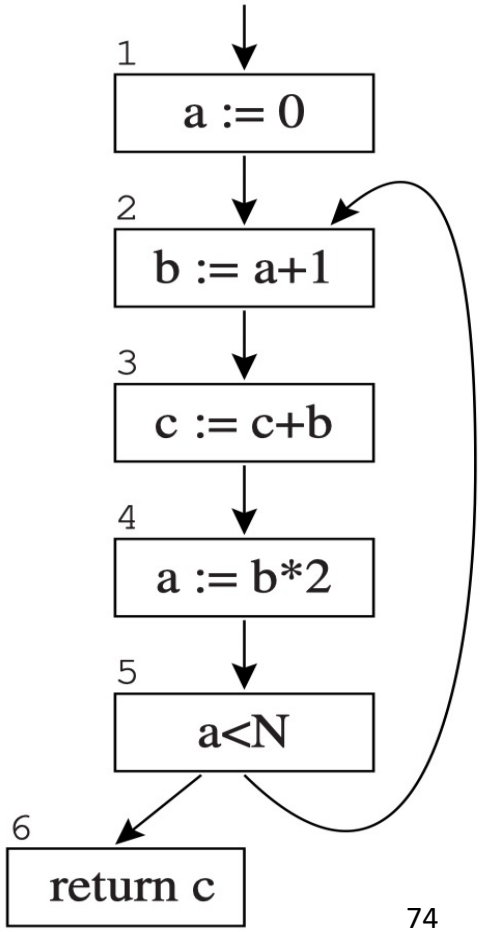
Example: Calculation of Liveness, Revisited

- out[n] is computed from in[s], in[n] is computed from out[n]
- **Strategy**: the opposite order (from 6 to 1, from out to in)

	use	def	1st out	1st in	2nd out	2nd in	3rd out	3rd in
6	c			c		c		c
5	a		c	ac	ac	ac	ac	ac
4	b	a	ac	bc	ac	bc	ac	bc
3	bc	c	bc	bc	bc	bc	bc	bc
2	a	b	bc	ac	bc	ac	bc	ac
1		a	ac	c	ac	c	ac	c

$$in[n] = use[n] \cup (out[n] - def[n])$$

$$out[n] = \bigcup_{s \in succ[n]} in[s]$$



Summary: Calculation of Liveness

- Following forward control-flow edges VS. Computing in the opposite order
- When solving dataflow equations by iteration, the order of computation should follow the "**flow**" of dataflow facts
- Liveness flows backward along control-flow arrows, and from "out" to "in", so should the computation.

3. More Discussions

- Improvements
- Theoretical Results
- Static vs. Dynamic Liveness

Optimizing the Iterative Solving Process

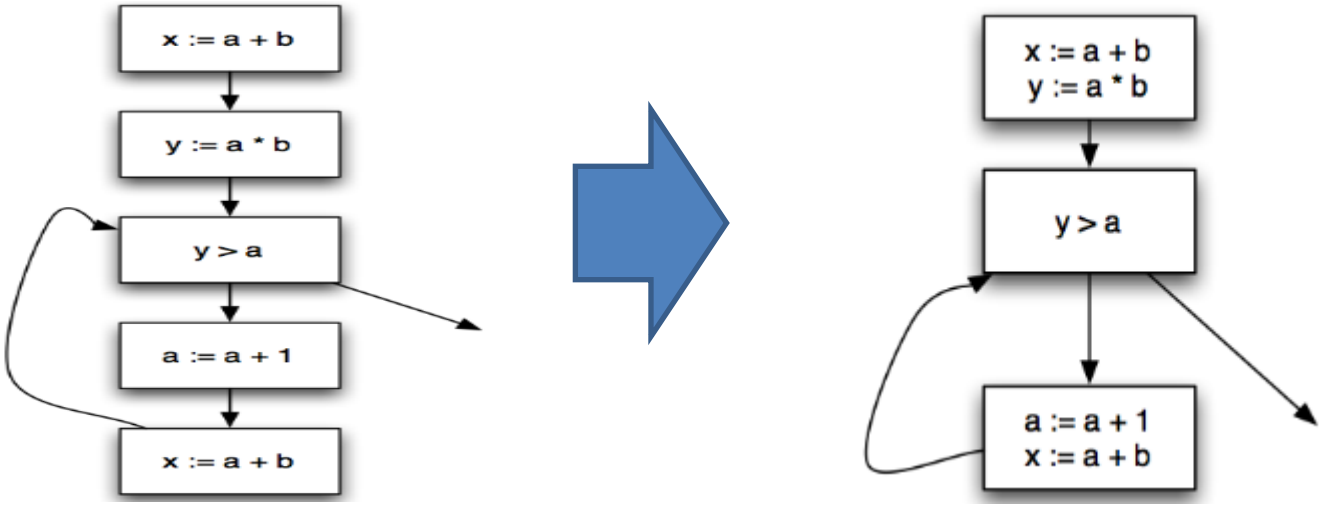
$$\text{in}[n] = \text{use}[n] \cup (\text{out}[n] - \text{def}[n])$$

$$\text{out}[n] = \bigcup_{s \in \text{succ}[n]} \text{in}[s]$$

- **Ordering the nodes**
- Use variants of Control-flow graph (CFG)
- Once a variable
- Careful selection of set representation
- Use other intermediate representation (IR)
- ….?

Improvements: Use Basic Bock-based CFG

- **Basic blocks**: Merge flow-graph nodes with only one predecessor and one successor
  - One entry point, one exit point
  - No branches except at the end



Benefits: Reduces graph size and speeds up dataflow analysis

Improvements: Use Basic Bock-based CFG

- **Block-level summaries:** For a basic block

  $B = [s_1, s_2, \ldots, s_k]$, scan **from last to first**:

Start: $use_B = \emptyset$, $def_B = \emptyset$
For $i = k$ down to 1:
  $use_B = use(s_i) \cup (use_B - def(s_i))$
  $def_B = def(s_i) \cup (def_B - use(s_i))$

Key insight: After pre-computing use_B and def_B once, the iterative solver works on the *block-level CFG* with exactly the same equations — just with B replacing individual nodes.

Improvements: Representations of Sets

Two common representations for in[n] and out[n]

Bit Arrays:

- Use N bits for N variables
- 1 bit = variable is in the set
- **Union** = bitwise OR
- **Set difference** = AND with complement
- Equality check = word-by-word compare

✅ **Good for dense sets**

O(N/w) per operation (w = word size, e.g. 64)

Sorted Lists:

- Linked list of set members
- Sorted by variable name/ID
- **Union** = merge sorted lists
- **Set difference** = linear scan
- Equality check = element-by-element

✅ **Good for sparse sets**

$O(|S_1|+|S_2|)$ per operation

When the sets are sparse, the sorted-list representation is faster.

Improvements: Variants of the Calculation

- **Alternative:** instead of computing all variables in parallel, compute liveness for **one variable at a time**
- For each temporary *t*:
  1. Find every **use site** of *t* in the CFG.
  2. From each use site, trace **backward** along CFG edges (DFS/BFS).
  3. At each visited node, mark *t* as **live**.
  4. **Stop** when you reach a node that **defines** *t* (the definition "kills" further propagation).
- Why fast
  - Many temporaries have **short live ranges** → DFS terminates quickly
  - Does **not** traverse entire flow graph for most variables

3. More Discussions

- Improvements
- Theoretical Results
- Static vs. Dynamic Liveness

Theorical Results: Time Complexity

- A program of size N: at most N nodes and at most N variables. Each set-union operation takes O(N) time.
- For loop: computes a constant number of set operations per node; there are O(N) nodes => O(N^2) time
- Repeat loop: The sum of the sizes of all in and out sets is 2N^2, which is the most that the repeat loop can iterate

- Worst-case run time:
  - **O(N^4)**
- In practice :
  - **Between O(N) and O(N^2)**
  - Proper computation order

```
for each n
  in[n] ←{}; out[n] ←{}
repeat
  for each n
    in′[n] ← in[n]; out′[n] ← out[n]
    in[n] ← use[n] ∪ (out[n] − def[n])
    out[n] ← ∪ s ∈succ[n] in[s]
until in′[n] = in[n] and out′[n] = out[n] for all n
```

Theorical Results: Least Fixed Points

- **Theorem.** Equation 10.3 have more than one solutions
  - *E.g.*, X and Y
- Indeed, any solution to the dataflow equations is a conservation approximation.

			X		Y		Z	
	use	*def*	*in*	*out*	*in*	*out*	*in*	*out*
1		a	c	ac	cd	acd	c	ac
2	a	b	ac	bc	acd	bcd	ac	b
3	bc	c	bc	bc	bcd	bcd	b	b
4	b	a	bc	ac	bcd	acd	b	ac
5	a		ac	ac	acd	acd	ac	ac
6	c		c		c		c	

TABLE 10.7. X and Y are solutions to the liveness equations; Z is not a solution.

$$in[n] = use[n] \cup (out[n] - def[n])$$
$$out[n] = \bigcup_{s \in succ[n]} in[s]$$

Theorical Results: Least Fixed Points

- **Theorem.** Equation 10.3 have more than one solutions
- If X is the solution and all solutions contains Solution X, we say that X is the **least solution (least fixed point)**
- **Theorem**. Equation 10.3 have a least fixed point, and the iteration algorithm described above always computes the least fixed point.

$$in[n] = use[n] \cup (out[n] - def[n])$$
$$out[n] = \bigcup_{s \in succ[n]} in[s]$$

Equation 10.3

```
for each n
  in[n] ←{}; out[n] ←{}
repeat
  for each n
    in′[n] ← in[n]; out′[n] ← out[n]
    in[n] ← use[n] ∪ (out[n] − def[n])
    out[n] ← ∪s ∈succ[n] in[s]
until in′[n] = in[n] and out′[n] = out[n] for all n
```

3. More Discussions

- Improvements
- Theoretical Results
- Static vs. Dynamic Liveness

Static and Dynamic Liveness

- **Static liveness (over-approximation)**
  - A variable *a* is statically live at node *n* if there is **some path of control-flow edges** from *n* to some use of *a* that does not go though a definition of *a*.
- **Dynamic liveness (under-approximation)**
  - A variable *a* is dynamically live at node *n* if **some execution** of the program goes from *n* to a use of *a* without going through any definition of *a*.

If a is dynamically live, it is also statically live.

Summary

- Liveness analysis determines which variables are needed in the future
- Conservative approximation is necessary
- Typically, formulated and solved using dataflow equations on control-flow graph
- Liveness flows backwards through program
- Iterative solution finds the least fixed point
- Algorithmic and empirical improvements

Thank you all for your attention